

Kubernetes OOMKilled Pod Recovery Runbook

Document ID: RB-K8S-002 | Version: 1.0 | Last Updated: 2024-11-15

Category	kubernetes	Severity	P1 - Critical
Domain	Container Orchestration	Est. Duration	20-40 minutes
Tags	kubernetes, oomkilled, memory, limits, containers		

Overview

OOMKilled (Out of Memory Killed) occurs when a container exceeds its memory limit and the Linux kernel OOM killer terminates the process. Exit code is 137. This runbook covers diagnosis, immediate mitigation by adjusting memory limits, and long-term remediation through application profiling.

Prerequisites

- kubectl configured and connected to the target cluster
- Access to Prometheus/Grafana for memory metrics
- Permission to edit Deployments in the affected namespace
- Helm (if deployment managed via Helm chart)

Response Steps

Step 1: Confirm OOMKilled Exit Code

Describe the pod and confirm the last state shows OOMKilled with exit code 137.

```
kubectl describe pod -n | grep -A10 'Last State'
kubectl get pod -n -o jsonpath='{.status.containerStatuses[0].lastState}'
```

NOTE: Exit code 137 = OOMKilled. Exit code 1 = application crash. Do not confuse the two.

Step 2: Check Current Memory Limits

Retrieve the current resource requests and limits for the affected deployment.

```
kubectl get deployment -n -o jsonpath='{.spec.template.spec.containers[*].resources}'
kubectl top pods -n --sort-by=memory
```

Step 3: Review Memory Metrics

Query Prometheus for the container's actual memory usage over the past 24 hours to determine the correct new limit.

```
kubectl port-forward svc/prometheus-operated 9090:9090 -n monitoring &
# Query: container_memory_working_set_bytes{namespace='',pod=~'.*'}

```

NOTE: Set the new limit to peak usage + 25% headroom.

Step 4: Patch Memory Limits

Update the deployment with a higher memory limit. Use kubectl patch for a targeted change without redeploying.

```
kubectl patch deployment -n --type=json -p='[{"op":"replace","path":"/spec/template/spec/co
ntainers/0/resources/limits/memory","value":"512Mi"}]'
```

```
kubectl patch deployment -n --type='json' -p='[{"op":"replace","path":"/spec/template/spec/containers/0/resources/requests/memory","value":"256Mi"}]'
```

Step 5: Monitor Pod Restart

Watch the rollout and confirm new pods start without OOMKilled.

```
kubectl rollout status deployment/ -n  
kubectl get pods -n -w
```

NOTE: SUCCESS: All pods Running with RESTARTS=0 for at least 5 minutes.

Step 6: Verify Memory Usage Post-Fix

Confirm actual memory usage is within the new limits after the fix.

```
kubectl top pods -n  
kubectl describe pod -n | grep -A5 'Limits'
```

Rollback Steps

Rollback Step 1: Revert Limit Patch

If the new limit causes resource starvation on the node, revert to the previous value.

```
kubectl rollout undo deployment/ -n
```

Rollback Step 2: Cordon Node if Node Pressure

If the node is under memory pressure, cordon it to prevent further scheduling.

```
kubectl cordon  
kubectl drain --ignore-daemonsets --delete-emptydir-data
```

Step Dependencies

Step	Depends On	Can Parallelize With
Step 1: Confirm OOMKilled	—	—
Step 2: Check Limits	Step 1	—
Step 3: Review Metrics	Step 1	Step 2
Step 4: Patch Limits	Step 2, Step 3	—
Step 5: Monitor Restart	Step 4	—
Step 6: Verify Post-Fix	Step 5	—

Maintained by Platform Engineering. For updates ping #platform-oncall.